

Chapter Abstract. MLA matters because attention-cache memory can dominate decode. Its defining object is the cache shape, not the attention-variant label. MHA, MQA, and GQA provide the baselines. Against them, MLA introduces the low-rank latent KV state, the non-RoPE/RoPE score split, and the decode-time representation needed by DeepSeek-style MLA. A small cache arithmetic example keeps storage and read volume separate from accuracy. Sparse attention and serving-kernel details are developed in later chapters.

4.1. Why Attention Cache Shape Matters

Autoregressive decode repeatedly reads attention state for earlier positions. As context and concurrency grow, the representation stored per position becomes a first-order systems object. MLA changes that representation; it does not change the causal obligation or make later runtime support automatic [26, 32]. The byte-accounting baseline belongs to Chapter 2; the runtime realization of the resulting cache contract belongs to the serving chapter in Volume II.

Table 4.1 contrasts causal visibility with the cache representation, while Table 4.2 records the symbols used in the chapter’s cache-shape examples.

Definition 4.1.1 (Autoregressive decode step) *At position t , a decoder-only transformer computes the next hidden state using model parameters, the current token representation, and cached attention state for positions $1, \dots, t - 1$. No computation for t may read a future position.*

The chapter’s running example begins after a four-position prompt. Decoding x_5 may read cached state for $x_1, \dots, x_4$, then appends state for x_5 . MHA, GQA, and MLA obey this same rule; they differ in what must be stored per earlier position.

Figure 4.1 holds those four causal positions fixed while changing only their stored representation. The visual comparison is qualitative: the exact tensor shapes and byte counts are derived later in the chapter.

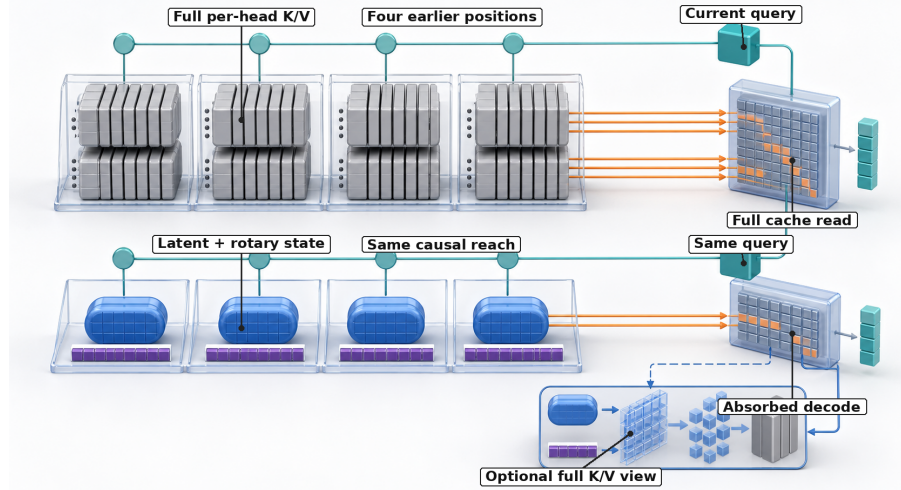
Rotary position embedding (RoPE) rotates only part of each query and key. Throughout this chapter, *non-RoPE* names the coordinates that are not rotated.

Question	Unchanged obligation	Representation choice
Which positions may be read?	Only earlier positions at decode step t	All variants use the causal mask
What is stored?	Enough information for scores and value aggregation	MHA stores per-head K, V ; GQA fewer KV heads; MLA (c_i, k_i^R)
Why does serving pressure change?	Every later token revisits retained state	Per-position payload controls repeatedly read bytes

Table 4.1.: MLA changes cache representation while preserving causal attention.

Figure 4.1.: The upper lane stores a full per-head key/value view for each of four earlier positions. The lower lane keeps one compact latent state and a separate RoPE-sensitive component per position while preserving the same causal reach. The inset is an optional materialized full key/value view; the absorbed decode path need not store that full view. Object sizes are schematic rather than a claimed compression ratio.

Table 4.2.: Local notation for MLA cache shape and decode examples.



Symbol	Meaning
t, h_t	Decode position and token state
c_i, k_i^R	Latent KV state and RoPE-sensitive state for position i
H_Q, H_{KV}	Query-head and key/value-head counts
$d_{\text{nope}}, d_{\text{rope}}, d_v$	Non-RoPE (source name NoPE), RoPE, and value widths
r	Low-rank latent dimension

DeepSeek’s configuration and implementation call this branch *NoPE*, as in `qk_nope_head_dim` and `q_nope`. This book uses *non-RoPE* in explanatory prose and reserves *NoPE* for that DeepSeek source label. In other positional-encoding work, NoPE can instead name a whole decoder-only Transformer with no explicit positional encoding; that model-wide usage is not the meaning here [98]. The non-RoPE branch may still receive position-aware hidden states from the surrounding model. The complementary RoPE branch is explicitly rotated and therefore remains tied to the positions being scored.

Take a toy layer with $T = 4$, $H_Q = 4$, and $d_k = d_v = 2$. A dense MHA cache read contains $TH_Q(d_k + d_v) = 64$ scalar elements, or 128 bytes in BF16.

Definition 4.1.2 (Cache read volume) *For one layer and decode step, cache read volume is the number of stored cache elements read from previous positions, multiplied by their storage byte width. It excludes other kernel traffic and end-to-end latency.*

The cache-read volume in Definition 4.1.2 makes the chapter’s central question precise: can the model retain less state per previous position while reconstructing the attention quantities required by the next causal decode step?

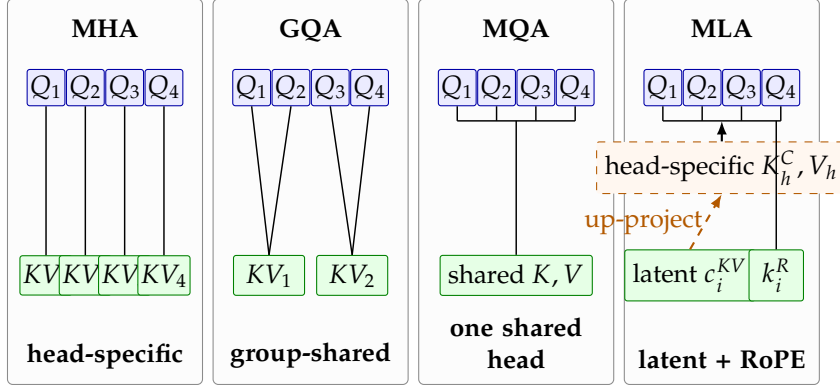


Figure 4.2: Four cache representations with the four query heads held fixed. MHA, GQA, and MQA reduce ordinary stored key/value heads by progressively sharing them. MLA changes the cached object: the persistent state is the compressed latent c_i^{KV} plus the decoupled RoPE key component k_i^R ; the dashed orange box is reconstructed work, not additional persistent cache. The plate compares representation structure; the exact toy arithmetic appears once in Table 4.3. It does not predict realized kernel speed.

4.2. MHA, MQA, and GQA as Cache Baselines

MLA is easiest to understand after fixing the baselines. Standard attention uses a query vector to score stored keys and then forms a weighted sum of stored values [134]. Multi-head attention repeats this calculation in several heads, allowing different projections of the same token stream to attend in different ways. MQA and GQA change how many key/value heads are stored and shared, and so they give a useful bridge from ordinary MHA to the more structural cache change made by MLA [126, 1].

Definition 4.2.1 (Multi-head attention cache) *An MHA cache for a layer with H_Q query heads has H_Q key heads and H_Q value heads. For each previous position i , it stores $(K_{i,h}, V_{i,h})$ for every head $h \in \{1, \dots, H_Q\}$, with $K_{i,h} \in \mathbb{R}^{d_k}$ and $V_{i,h} \in \mathbb{R}^{d_v}$.*

Each cached token has one key vector and one value vector for every query head. The attention score for head h at token x_5 uses the head-specific query $q_{5,h}$ and the stored key $K_{i,h}$ for each old position i . The output for the same head uses the stored values $V_{i,h}$. The important point is that every query head carries its own key/value cache.

Definition 4.2.2 (Multi-query attention cache) *An MQA cache for a layer with H_Q query heads has one key head and one value head shared by all query heads. For each previous position i , it stores a single pair (K_i, V_i) , and each query head scores against that same stored key and aggregates from that same stored value.*

The four query heads do not disappear under MQA; the queries are still separate. What changes is the number of key/value heads that must be stored and read from the cache. This is why MQA can be described as one write-head for decoding: the cache write path produces one shared key/value head, and many query heads read against it [126].

Figure 4.2 holds the four query heads fixed and changes only the representation that each old token leaves in the cache. The first three panels show head sharing. The fourth previews MLA’s different move: cache a compressed latent and a decoupled RoPE key component, then reconstruct head-specific views when attention runs [26].

Definition 4.2.3 (Grouped-query attention cache) *A GQA cache for a layer with H_Q query heads has H_{KV} key/value heads, where $1 < H_{KV} < H_Q$ and the query heads are partitioned into groups. Query heads in the same group share one stored key head and one stored value head.*

Take $H_Q = 4$ and $H_{KV} = 2$. Heads 1 and 2 can share the first KV head, while heads 3 and 4 share the second KV head. GQA is therefore a controlled sharing scheme between MHA's per-head storage and MQA's single shared head: it reduces cache state by reducing the number of stored key/value heads, but it still stores ordinary key and value vectors for those heads.

MHA, GQA, and MQA keep the query-head count fixed while changing how many ordinary KV heads are stored.

The exact cache-shape arithmetic for the four representations is centralized in Table 4.3. Those counts do not imply that a real GPU kernel speeds up in the same ratio: kernel fusion, page layout, batch shape, and dtype conversion still affect realized performance. The structural baseline matters because MLA changes both KV-head storage and the representation retained for each previous position.

4.3. Low-Rank Latent KV State

MLA replaces the ordinary assumption that the cache must store all attention-compatible key and value coordinates directly. DeepSeek-V2 describes MLA as using low-rank key-value union compression to address the inference-time KV-cache bottleneck, and DeepSeek-V3 continues to use MLA in the architecture family [25, 26, 32]. The word “latent” is important: the stored vector is not itself the full per-head key or value. It is a smaller state from which the computation can obtain the information needed for scores and outputs.

Concept-to-code map

The pinned implementation path is <https://github.com/deepseek-ai/DeepSeek-V3/blob/9b4e9788e4a3a731f7567338ed15d3ec549ce03b/inference/model.py>.

- **MLA motivation and mechanism.** The low-rank KV-compression claim maps to the DeepSeek-V2 technical report and model description.
- **V3 continuity.** The retained MLA architecture maps to the pinned DeepSeek-V3 model record and configuration cited above.

Definition 4.3.1 (Low-rank latent KV representation) *A low-rank latent KV representation stores, for each token position i , a latent vector $c_i \in \mathbb{R}^r$. Learned projection matrices map c_i into the key and value components required by the layer's attention calculation. The number r is the latent width. It is an upper bound on the rank of each induced reconstruction map, not a promise that every learned matrix has exact rank r .*

Set $r = 3$. Instead of storing all four heads' ordinary keys and values, the cache stores a three-coordinate vector c_i for each old position, plus a separate RoPE-sensitive component introduced later. The definition does not require the reader to know DeepSeek's exact implementation yet. It says only that the cache stores a lower-dimensional state and that the attention computation is written so that this state is sufficient.

An MLA cache entry stores c_i plus the RoPE-sensitive key part, not a full per-head key/value cache.

A small matrix exposes the rank constraint. Suppose an ordinary value quantity for one position would be a vector in $\mathbb{R}^4$, but the model stores $c_i \in \mathbb{R}^2$ and uses a learned matrix $W_V \in \mathbb{R}^{4 \times 2}$ to produce $v_i = W_V c_i$. If

$$W_V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad c_i = \begin{bmatrix} 2 \\ 3 \end{bmatrix},$$

then $v_i = (2, 3, 5, -1)^\top$. The stored state has two scalars, while the materialized value has four. The model has not compressed an already-computed dense cache after the fact; it has parameterized the layer so that the stored state and the projections are part of the attention computation.

Example 4.3.1 (A learned latent subspace) The two columns of W_V above span every value vector that this simplified decoder can reconstruct. Thus every representable vector has the form

$$(a, b, a + b, a - b)^\top$$

for some latent coordinates a, b . The vector $(0, 0, 1, 0)^\top$ is outside that column space: its first two coordinates would force $a = b = 0$, which would also force its third coordinate to be zero. The two-coordinate latent state therefore cannot reproduce an arbitrary four-coordinate cache vector losslessly. Training instead learns the compression and reconstruction maps together so that task-useful key/value information is carried by the learned subspace. Increasing r enlarges the available capacity but also increases the retained state and decode traffic.

Definition 4.3.2 (MLA cache entry) *An MLA cache entry for a previous position consists of the latent KV vector c_i together with any separately stored position-sensitive key component needed for the attention score. In this chapter's notation the entry is (c_i, k_i^R) , where $c_i \in \mathbb{R}^r$ and $k_i^R \in \mathbb{R}^{d_{\text{rope}}}$.*

With $r = 3$ and $d_{\text{rope}} = 1$, the MLA cache entry in Definition 4.3.2 retains the latent coordinates plus the separate positional component for every cached position. The ledger below evaluates that stored form against the three ordinary key/value baselines. Under real dimensions, the ratio depends on the public configuration fields and on the kernel layout used to store the latent and positional components.

Figure 4.2 is the authoritative visual for what each representation retains and what MLA reconstructs. The ledger below then carries the exact four-position scalar and byte arithmetic, avoiding a second footprint graphic with the same conceptual duty.

Table 4.3 puts all four cache representations on one ledger. In the toy dimensions, MQA and the simplified MLA entry have the same scalar count, but they get there by different mechanisms. MQA stores one ordinary key/value head and shares it across query heads. MLA stores a latent KV state plus the positional component needed for scoring, then arranges the score and output calculations around that stored form.

Table 4.3.: Toy tensor-shape ledger for dense, shared-head, and latent cache representations. The ledger separates what is stored, what view the attention calculation obtains, what decode reads, and what still participates in score and output multiplication.

scheme	stored cache entry per position	reconstructed, shared, or absorbed view	read across $T = 4$ positions	multiplied during attention
MHA	$\{K_{i,h}, V_{i,h}\}_{h=1}^{H_Q}$, 16 scalars	The stored tensors are already the per-head key/value view.	$4 \cdot 16 = 64$ scalars, 128 bf16 bytes	Each $q_{t,h}$ scores $K_{i,h}$; weights aggregate $V_{i,h}$.
MQA	One shared (K_i, V_i) , 4 scalars	The single KV head is read by all $H_Q = 4$ query heads.	$4 \cdot 4 = 16$ scalars, 32 bf16 bytes	Each $q_{t,h}$ scores the same K_i ; weights aggregate the same V_i .
GQA	$\{K_{i,g}, V_{i,g}\}_{g=1}^{H_{KV}}$, 8 scalars	The group map chooses which stored KV head each query head reads.	$4 \cdot 8 = 32$ scalars, 64 bf16 bytes	Head h scores $K_{i,g(h)}$; weights aggregate $V_{i,g(h)}$.
MLA, simplified	(c_i, k_i^R) , $r + d_{\text{rope}} = 4$ scalars	Non-RoPE key/value information comes from c_i ; k_i^R stays position-bound.	$4(r + d_{\text{rope}}) = 16$ scalars, 32 bf16 bytes	Transformed query and RoPE terms still score retained positions; output uses a weighted latent path plus projection.

Two negative statements prevent common misunderstandings. First, the latent state is not an attention probability. The probabilities are still produced by normalizing scores over positions. Second, MLA is not the same as sparse attention. In the example above, token x_5 can still attend to all four previous positions; what changed is the representation stored for each position. Sparse attention, discussed later in the chapter, changes which positions are selected.

4.4. Simplified MLA Projection Paths

The projection path is where MLA stops being only a cache-size slogan. A decoder block receives a hidden vector, projects it into query information for the current token, projects earlier hidden vectors into latent KV information, and then arranges the score and value computations so that the cache can store the latent state. DeepSeek-V3’s public inference code exposes this structure through an MLA module with query low-rank fields, KV low-rank fields, non-RoPE/RoPE dimensions, and two attention implementation modes [29].

Let $h_i \in \mathbb{R}^{d_{\text{model}}}$ be the layer input for position i . A simplified MLA block can be written as

$$c_i = A_{kv} h_i, \quad \begin{bmatrix} k_i^N \\ v_i \end{bmatrix} = B_{kv} c_i, \quad q_i = B_q A_q h_i,$$

where $c_i \in \mathbb{R}^r$, k_i^N is the non-rotary key component, and v_i is the value component used after the attention weights are known. This notation hides several engineering

details, such as normalization and tensor parallel partitioning, but it captures the logical path: the cache can store c_i rather than the full materialized pair (k_i^N, v_i) .

Suppose $h_i \in \mathbb{R}^6$, $r = 3$, $d_{\text{nope}} = 2$, and $d_v = 2$. The matrix A_{kv} has shape 3×6 , and the matrix B_{kv} has shape 4×3 . The first two rows of $B_{kv}c_i$ can supply a non-rotary key part, and the last two rows can supply a value part. The ordinary materialized object has four coordinates, but the cache stores three latent coordinates before adding the RoPE-sensitive part. This is the same mathematical economy as the rank example in the previous section, now placed inside the attention block.

Definition 4.4.1 (Absorbed projection) *An absorbed projection is an algebraic arrangement in which a projection that would otherwise materialize a full key or value representation is moved into the query-side or output-side computation, so that attention can use the latent cache state without storing the full per-head key/value cache.*

The definition rests on ordinary associativity of compatible matrix products. Let $q \in \mathbb{R}^2$, $W \in \mathbb{R}^{2 \times 3}$, and $c \in \mathbb{R}^3$. The score

$$q^\top (Wc)$$

can be computed as

$$(W^\top q)^\top c.$$

For a numeric instance,

$$q = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0 & 3 \\ 2 & 1 & 1 \end{bmatrix}, \quad c = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}.$$

Materializing the key gives $Wc = (22, 19)^\top$, and the score is $1 \cdot 22 + 2 \cdot 19 = 60$. Absorbing the projection gives $W^\top q = (5, 2, 5)^\top$, and the score is $5 \cdot 4 + 2 \cdot 5 + 5 \cdot 6 = 60$. The same scalar score is obtained, but the cached state can remain c . The value path uses the same linearity in the other direction. If a materialized value has the form $v_j = B_v c_j$, then attention weights α_j satisfy

$$\sum_j \alpha_j v_j = \sum_j \alpha_j B_v c_j = B_v \left(\sum_j \alpha_j c_j \right).$$

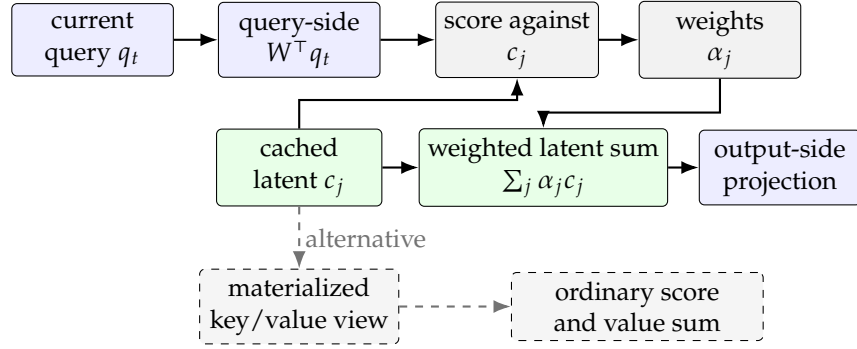
The cache saving comes from keeping per-position state as c_j , not from skipping the later output-side projection work.

Projection-Path Checkpoint Absorption does not delete the second projection; it changes where the projection is paid. Read the path as three objects: the cached latent state c_j , the score computation that can compare a transformed query with c_j , and the output-side value projection that still reconstructs the value information needed after softmax. The memory saving belongs to the cache object, while the arithmetic obligation remains in the score and output paths.

Figure 4.3 shows this rearrangement as a cache-layout distinction. The faded route stands for a path that materializes a larger key/value view before attention. The

Projection absorption changes where multiplication occurs; it does not remove the projection from the computation.

Figure 4.3.: Absorbed MLA projection path. The solid route scores and aggregates against cached latent state, then applies the output-side projection. The dashed alternative first materializes a full key/value view.



emphasized route keeps compact latent cache entries as the state read by decode and moves compatible projection work around the score and output calculations.

The DeepSeek-V3 public configuration gives concrete names to these dimensions. For the 671B inference config, ‘n_heads’ is 128, ‘q_lora_rank’ is 1536, ‘kv_lora_rank’ is 512, ‘qk_nope_head_dim’ is 128, ‘qk_rope_head_dim’ is 64, and ‘v_head_dim’ is 128 [29]. In this chapter’s notation, the field ‘kv_lora_rank’ supplies the public analogue of r , while ‘qk_nope_head_dim’, ‘qk_rope_head_dim’, and ‘v_head_dim’ correspond to d_{nope} , d_{rope} , and d_v . The code names ‘wq_a’, ‘wq_b’, ‘wkv_a’, and ‘wkv_b’ record the same split into first-stage and second-stage projections.

chapter notation	DeepSeek-V3 public field or object
H_Q	n_heads
r	kv_lora_rank
d_{nope}	qk_nope_head_dim
d_{rope}	qk_rope_head_dim
d_v	v_head_dim
A_q, B_q	wq_a, wq_b
A_{kv}, B_{kv}	wkv_a, wkv_b

This mapping is source support, not a substitute for the mathematical explanation. The latent state and absorbed projection come first; the public fields then identify where the design appears in a real model. That order is important because the same field names are also entangled with tensor parallelism, normalization, and inference-specific choices that would obscure the main idea if introduced too early.

The public dimensions also support a compact executable byte ledger. For one token and one layer, the checkpoint compares a hypothetical cache that stores keys and values for every head with the absorbed latent-plus-RoPE form. It computes representation bytes, not allocator use, memory traffic, or latency.

Executable checkpoint: V3 cache representation bytes

```

1 n_heads = 128
2 qk_nope_width = 128
3 qk_rope_width = 64
4 value_width = 128
5 kv_latent_width = 512
6 bytes_per_value = 2 # BF16
7
8 per_head_width = qk_nope_width + qk_rope_width + value_width
9 materialized_values = n_heads * per_head_width
10 mla_values = kv_latent_width + qk_rope_width
11 materialized_bytes = materialized_values * bytes_per_value
12 mla_bytes = mla_values * bytes_per_value
13 storage_ratio = materialized_bytes / mla_bytes
14
15 assert materialized_values == 40_960
16 assert mla_values == 576
17 assert (materialized_bytes, mla_bytes) == (81_920, 1_152)
18 assert round(storage_ratio, 2) == 71.11
19
20 if __name__ == "__main__":
21     print(
22         f"cache bytes: materialized={materialized_bytes:}, ",
23         f"MLA={mla_bytes:}, ratio={storage_ratio:.2f}x"
24     )

```

Run: `python3 chapter_multi_head_latent_attention/mla_v3_cache_checkpoint.py`

The assertions recover the chapter's scalar counts: 40,960 materialized values versus 576 retained MLA values, or 81,920 versus 1,152 BF16 bytes. Their 71.11-fold ratio is therefore a storage-representation ratio only. The directly executable file lives at `chapter_multi_head_latent_attention/`.

The absorption step is ordinary associativity with a shape obligation. If a cached latent row $c_j \in \mathbb{R}^r$ would first expand through $W^K \in \mathbb{R}^{r \times d_k}$ and then meet query $q_i \in \mathbb{R}^{d_k}$, the score is $q_i(W^K)^T c_j^T$. Reassociate it as $(q_i(W^K)^T) c_j^T$: the query becomes an r -wide vector and can score the retained latent row directly. This equality avoids expansion on the linear branch. That branch must be position-independent; the equation does not absorb a position-dependent rotation.

4.5. RoPE Interactions and the NoPE/RoPE Split

Rotary position embedding changes the attention score by making part of the query and key depend on token position [129]. This matters for MLA because not every component can be treated as a position-independent latent vector. If the score for token x_5 compares a query component at position 5 with key components for positions 1, ..., 4, the position phases in the score must correspond to the positions being compared. DeepSeek-V3 exposes this separation through public fields for non-rotary and rotary query-key dimensions, and its inference implementation splits the query and KV paths accordingly [29]. The section title follows the

DeepSeek-V3 implementation’s paired NoPE/RoPE naming. Here and in the rest of the book, that NoPE label applies only to the non-RoPE branch described below, not to a model-wide absence of positional encoding.

Definition 4.5.1 (RoPE on one coordinate pair) *For a base angle θ , RoPE rotates each two-coordinate query or key pair at position t by*

$$R_t(\theta) = \begin{bmatrix} \cos(t\theta) & -\sin(t\theta) \\ \sin(t\theta) & \cos(t\theta) \end{bmatrix}.$$

If unrotated pair vectors are q and k , their positioned dot product is

$$\langle R_t q, R_i k \rangle = q^\top R_{i-t} k.$$

The absolute rotations cancel into the relative offset $i - t$. A real rotary width contains multiple coordinate pairs, generally with different base angles [129].

Take $q = k = (1, 0)^\top$ and $\theta = \pi/2$. The same-position score is one. A key one position away rotates to $(0, 1)^\top$, giving score zero; a key two positions away rotates to $(-1, 0)^\top$, giving score minus one. The content vectors did not change. Only the relative position changed the dot product. This is the geometric reason that a rotary key remains tied to the position at which it was created.

Table 4.4: A two-dimensional RoPE trace. Rotation turns absolute position tags into a dot product that depends on relative position.

Relative offset $i - t$	Positioned key pair	Dot product with $q = (1, 0)$
0	$(1, 0)$	1
1	$(0, 1)$	0
2	$(-1, 0)$	-1

Table 4.4 supplies the missing geometric step behind the position-tagged cache state used below.

Definition 4.5.2 (DeepSeek NoPE (non-RoPE) and RoPE components) *For the attention score in this chapter, q^N and k^N denote the non-RoPE query and key components: these coordinates do not receive the explicit RoPE rotation. The symbols q^R and k^R denote the rotary-positioned query and key components. The score between current position t and earlier position i combines the non-RoPE part with the position-sensitive RoPE part. In this definition, NoPE is the DeepSeek source label for the non-RoPE branch; it is not the model-wide NoPE architecture studied in positional-encoding work [98]. Neither the surrounding hidden states nor the model as a whole are position-free.*

A simplified score for head h can be written as

$$s_{t,i,h} = \langle q_{t,h}^N, k_{i,h}^N \rangle + \langle q_{t,h}^R, k_i^R \rangle.$$

The first inner product can be arranged through the latent state. The second inner product carries the position-dependent rotary information.

Example 4.5.1 (One score, two branches) Let $q^N = (1, 2)^\top$ and $k^N = (3, 1)^\top$. Their non-RoPE contribution is

$$\langle q^N, k^N \rangle = 1 \cdot 3 + 2 \cdot 1 = 5.$$

For the RoPE branch, let the unrotated pair vectors be $q^R = k^R = (1, 0)^\top$ and let $\theta = \pi/2$. If the cached key is one position behind the query, its RoPE contribution is $\cos(-\pi/2) = 0$, so the combined score is $5 + 0 = 5$. If the same key content is two positions behind, the RoPE contribution is $\cos(-\pi) = -1$, so the combined score is $5 - 1 = 4$. The non-RoPE term stays at 5; only the explicitly rotated branch changes with the relative position.

The same split is short enough to execute without hiding the rotation inside an attention library. The checkpoint computes the non-rotary inner product once, rotates the paired key at each relative offset, and retains both score components in its result.

Executable checkpoint: One NoPE/RoPE score split

```

1 from math import cos, pi, sin
2
3
4 def dot(left: tuple[float, ...], right: tuple[float, ...]) -> float:
5     return sum(a * b for a, b in zip(left, right, strict=True))
6
7
8 def rotate(pair: tuple[float, float], angle: float) -> tuple[float,
9     float]:
10     c, s = cos(angle), sin(angle)
11     return (c * pair[0] - s * pair[1], s * pair[0] + c * pair[1])
12
13 q_nope, k_nope = (1.0, 2.0), (3.0, 1.0)
14 q_rope = k_rope = (1.0, 0.0)
15 theta = pi / 2
16
17
18 def split_score(relative_offset: int) -> tuple[float, float, float]:
19     nope = dot(q_nope, k_nope)
20     rope = dot(q_rope, rotate(k_rope, relative_offset * theta))
21     return nope, rope, nope + rope
22
23
24 def rounded(values: tuple[float, ...]) -> tuple[float, ...]:
25     return tuple(round(value, 12) for value in values)
26
27
28 one_behind = split_score(-1)
29 two_behind = split_score(-2)
30 one_behind_rounded = rounded(one_behind)
31 two_behind_rounded = rounded(two_behind)
32
33 assert one_behind_rounded == (5.0, 0.0, 5.0)
34 assert two_behind_rounded == (5.0, -1.0, 4.0)
35
36 if __name__ == "__main__":
37     print(
38         "scores (NoPE, RoPE, total): "
39         f"one-behind={one_behind_rounded},
40         two-behind={two_behind_rounded}"
41     )

```

Run: `python3 chapter_multi_head_latent_attention/nope_rope_score_checkpoint.py`

The assertions keep the non-RoPE contribution at 5, while the rotary contribution changes from 0 to -1, producing the combined scores 5 and 4. The file is also checked from `chapter_multi_head_latent_attention/`. It executes one coordinate pair and the chapter's relative-position invariant, not the full MLA projection or a production RoPE layout.

In the running cache ledger, $d_{\text{rope}} = 1$ is one scalar-equivalent bookkeeping slot, not a literal one-dimensional rotation; actual RoPE operates on coordinate pairs. Each old token retains that abstract rotary key component k_i^R in addition

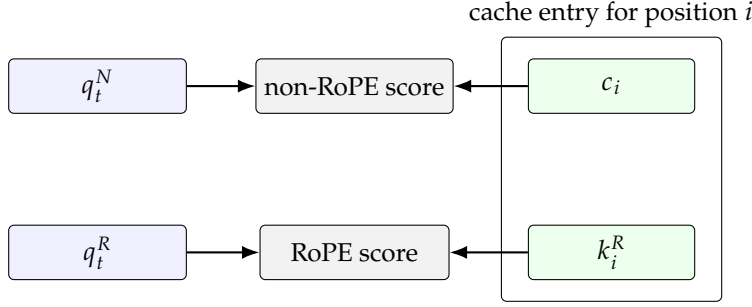


Figure 4.4: MLA cache entries keep latent state and rotary key state separate so the non-rotary and position-sensitive score terms can be reconstructed with the right position tags.

to its three-scalar latent vector c_i . The score for x_5 therefore uses q_5^R against $k_1^R, k_2^R, k_3^R, k_4^R$, each tagged by the earlier token’s position. Figure 4.4 shows the same split as a cache entry: the latent part can serve the non-rotary score path, while the rotary key remains position-addressed.

Invariant 4.5.3 (Positional scoring invariant). *At decode position t , every position-sensitive term used to score a previous token i must be the term associated with position i , and the query-side position-sensitive term must be the term associated with position t .*

This rule is simple, but it rules out an invalid compression shortcut. If the cache stored only an undifferentiated latent vector and lost the position-sensitive key component, the score would no longer have access to the proper rotary comparison for each previous position. The latent vector c_2 and the rotary component k_2^R both belong to position 2. When the model computes the score from x_5 to x_2 , it must combine the current query’s rotary part q_5^R with the stored position-2 component k_2^R , not with a component from another position.

The public DeepSeek-V3 configuration fixes the split at realistic scale. For the 671B config, the non-rotary query-key dimension is 128, and the rotary query-key dimension is 64 [29]. Thus the query-key scoring dimension in the public artifact is the sum of a non-rotary component and a rotary component. The implementation’s forward path reflects the same idea by splitting the query into ‘q_nope’ and ‘q_pe’ and the KV projection output into latent ‘kv’ and ‘k_pe’ components.

The toy dimensions are intentionally smaller but follow the same structure. With $d_{\text{nope}} = 2$, $d_{\text{rope}} = 1$, and $r = 3$, each old token has a three-scalar latent part and one scalar-equivalent rotary bookkeeping slot. The numerical cache count is therefore four: three latent scalars plus one rotary scalar-equivalent. That additional scalar preserves position-correct scoring while the key/value information remains in latent form.

Three MLA Obligations MLA carries three obligations at once. The latent cache state says what is stored for a previous token. The absorbed projection algebra says where a linear map can be associated without changing the score. The non-RoPE/RoPE split says which scoring component still carries position-sensitive information. A small cache example is therefore not a backend claim, an absorbed projection is not a position-encoding rule, and the rotary component is not the whole cached value.

The obstruction is that RoPE depends on both query and key positions. A static weight product can absorb fixed linear projections, but it cannot precompute the

Non-RoPE terms can use the latent path; RoPE terms must stay tied to the positions they score.

relative rotation $R_i^\top R_j$ for every future pair (i, j) . MLA therefore keeps a non-RoPE branch whose projections can be reassociated and a smaller RoPE branch whose positional state remains explicit. The model still uses positional encoding; the label non-RoPE applies only to the complementary query/key coordinates.

4.6. Decode-Time Cache Representation

The decode cache representation determines what gets appended after the current token and what gets read by the next token. A materialized MHA view stores ordinary per-head keys and values. An MLA-aware view stores the latent state and the position-sensitive key component, then arranges the score and output calculation around that stored form. DeepSeek-V3's public inference implementation makes this contrast explicit: a naive path has 'k_cache' and 'v_cache' buffers, while an absorbed path has 'kv_cache' and 'pe_cache' buffers [29].

Definition 4.6.1 (Materialized cache) *A materialized attention cache stores the key and value tensors in the same form in which the attention score and value aggregation read them. In an MHA materialized cache, the stored entries are per-head keys and per-head values.*

Definition 4.6.2 (Absorbed MLA cache) *An absorbed MLA cache stores the latent KV vector and the required position-sensitive key component, while leaving some key/value projection work to the query-side scoring path or output-side aggregation path.*

Materialized and absorbed caches can represent the same attention map while storing different per-position objects.

After token x_5 passes through the layer, an MHA cache appends four key vectors and four value vectors, GQA appends one key/value pair per KV group, and MLA appends (c_5, k_5^R) . The next token, x_6 , reads five retained positions rather than four. The ledger below is the single nearby source for the resulting append, read, and bf16 byte counts.

Figure 4.5 keeps the append shape and the next-token read shape separate.

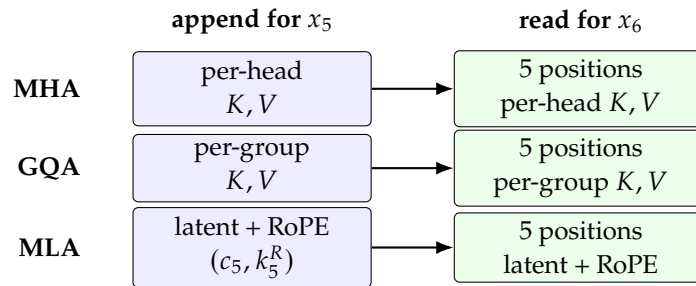


Figure 4.5.: The cache entry appended for x_5 determines the per-position payload that the next decode step reads across five retained positions. Exact quantities are centralized in the following ledger.

representation	append for x_5	read for x_6	bf16 bytes read
MHA	16 scalars	80 scalars	160
GQA, $H_{KV} = 2$	8 scalars	40 scalars	80
MLA, $r = 3, d_{\text{rope}} = 1$	4 scalars	20 scalars	40

The byte counts use bf16 and cover only the cache entries in this four-position trace. Real implementations may store scale factors, padding, page-block metadata, quantized bytes, or other layout details. For example, the FlashMLA interface documents explicit KV-cache layouts for MLA kernels, including an FP8-with-scale format that stores quantized non-RoPE data (called NoPE in FlashMLA), scale factors, and a bf16 RoPE part for DeepSeek-family shapes [60]. That implementation detail belongs mostly to the later kernel chapter, but it confirms the architectural point made here: MLA changes the cache-layout contract that the attention backend must satisfy.

Append shape and read shape can differ, so config parsing alone is not enough. A runtime can read `kv_lora_rank` and `qk_rope_head_dim` correctly, yet still allocate a cache layout that materializes the wrong tensors or calls an attention backend that expects ordinary MHA keys and values. Correct support has to connect the model config, the forward path, the cache allocation, the append operation, and the attention kernel. The concept-to-code map near the end of the chapter returns to this checklist.

The architecture-level cache obligation is smaller than a complete serving handoff. For every logical position i , the latent row c_i and RoPE-sensitive row k_i^R must remain paired, and the next step must append the newly projected pair without materializing an incompatible full per-head cache. Request ownership, physical page slots, publication order, and backend selection are runtime responsibilities developed in Volume II by the serving chapter, the FlashMLA chapter, and the systems correctness chapter.

Example 4.6.1 (A two-head absorbed decode with a rotary-row fault) *Setup.* This separate micro-trace uses one literal RoPE coordinate pair rather than the running ledger’s one scalar-equivalent rotary slot. Let $T = 3$, $t = 4$, $H_Q = 2$, and $r = d_{\text{nope}} = d_{\text{rope}} = d_v = 2$. The cached rows are

$$C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad K^R = \begin{bmatrix} 0 & 1 \\ -1 & 0 \\ 0 & -1 \end{bmatrix}.$$

The rows of K^R are $R_i(\pi/2)(1, 0)^\top$ for $i = 1, 2, 3$. At $t = 4$, take unabsorbed non-RoPE queries $q_1^N = q_2^N = (1, 0)^\top$ and positioned rotary queries $q_1^R = (1, 0)^\top$, $q_2^R = (0, 1)^\top$. Let W_h^{UK} and W_h^{UV} denote the head- h key and value slices of the latent up-projection, and let W_h^O denote the corresponding block of the output projection. For the key slices, choose

$$W_1^{UK} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_2^{UK} = \begin{bmatrix} -1 & 1 \\ 1 & 0 \end{bmatrix}.$$

Key-side absorption gives

$$\tilde{q}_h^N = (W_h^{UK})^\top q_h^N, \quad \tilde{q}_1^N = (1, 1)^\top, \quad \tilde{q}_2^N = (-1, 1)^\top.$$

For every cached latent row, associativity then preserves the non-RoPE score:

$$(q_h^N)^\top (W_h^{UK} c_i) = (\tilde{q}_h^N)^\top c_i.$$

Thus the full per-head non-RoPE key $W_h^{UK} c_i$ need not be materialized [26, 29]. *Score path.* The non-RoPE and rotary score contributions, scaled logits, and attention weights are

$$S^N = \begin{bmatrix} 1 & 1 & 2 \\ -1 & 1 & 0 \end{bmatrix}, \quad S^R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & -1 \end{bmatrix}.$$

$$L = \frac{S^N + S^R}{2} = \begin{bmatrix} 0.5 & 0 & 1 \\ 0 & 0.5 & -0.5 \end{bmatrix},$$

$$A = \text{softmax}(L) \approx \begin{bmatrix} 0.307 & 0.186 & 0.506 \\ 0.307 & 0.506 & 0.186 \end{bmatrix}.$$

Value and output path. Choose value up-projection slices and output blocks

$$W_1^{UV} = W_2^{UV} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_1^O = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}, \quad W_2^O = \begin{bmatrix} 1 & 0 \\ 1 & -2 \end{bmatrix}.$$

Absorbing each value slice into its output block gives

$$\hat{W}_h^O = W_h^O W_h^{UV}, \quad \hat{W}_1^O = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad \hat{W}_2^O = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

With $m_h = A_{h,:} C = \sum_i A_{h,i} c_i$, the two output contributions are

$$o_1 = \hat{W}_1^O m_1 \approx (0.814, 1.386), \quad o_2 = \hat{W}_2^O m_2 \approx (1.186, -0.199).$$

These equal the materialized value/output route because $\hat{W}_h^O = W_h^O W_h^{UV}$ and $m_h = \sum_i A_{h,i} c_i$, so

$$W_h^O \sum_i A_{h,i} W_h^{UV} c_i = \hat{W}_h^O m_h.$$

Fault and consequence. Swap only k_2^R and k_3^R , leaving c_2 and c_3 in place. Every cache row still has width $r + d_{\text{rope}} = 4$, but both attention rows become $(1/3, 1/3, 1/3)$. The outputs change to $\tilde{o}_1 = (2/3, 4/3)$ and $\tilde{o}_2 = (4/3, 0)$. Shape validity therefore does not imply positional alignment.

Figure 4.6 makes the numerical dependencies explicit. Its solid route is the absorbed computation, the dashed equality records equivalence to the materialized value/output route, and the dash-dotted route isolates the position-coupling fault.

The fault in the plate permutes one field inside otherwise legal cache rows. Shape and dtype checks cannot detect it because the failure changes the pairing between logical position and positional state rather than either tensor's extent. A physical cache test therefore needs the same mathematical oracle, but its block tables, request ownership, page slots, and publication evidence belong to Volume II.

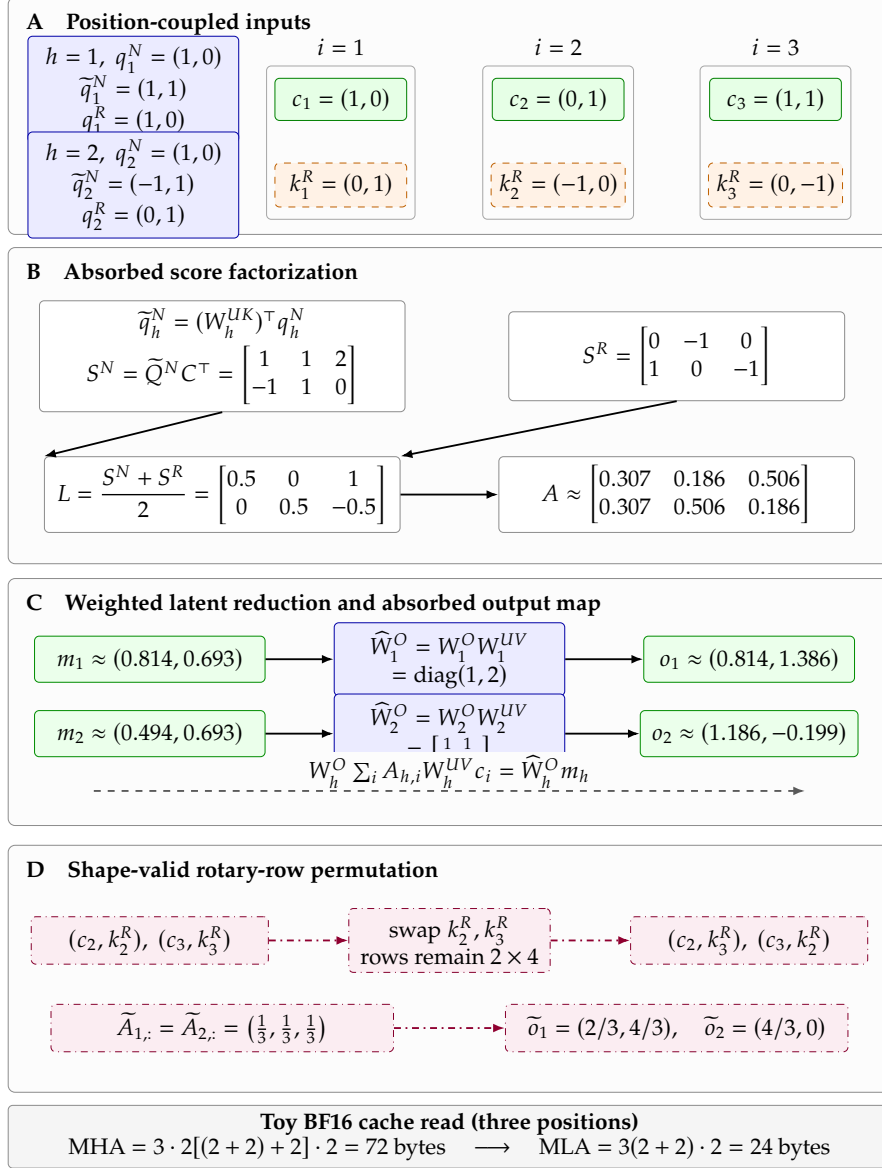


Figure 4.6: One absorbed MLA decode step at $t = 4$. Key-side and output-side absorption reproduce the materialized result while the latent and rotary cache fields remain coupled by logical position. Swapping only the rotary fields preserves tensor shapes but changes the normalized weights and outputs. The byte footer compares representation size for the toy trace, not kernel or serving latency.

One decode step can be read in four phases. First, append the new token’s latent KV row and rotary key state at its logical position. Second, project the current query into the absorbed latent-compatible and rotary branches. Third, score the query against earlier logical positions while pairing each latent row with the rotary state at the same position. Fourth, aggregate the value-compatible result and return one model-width row. Cache allocation may rearrange physical slots, but this logical append–pair–score–aggregate sequence is the reference behavior it must preserve.

4.7. Accuracy, Cost, and Serving Tradeoffs

The cache ledger establishes a representation result: under its declared dimensions, MLA retains fewer bytes per position than materialized multi-head keys and values. Model quality and serving speed are different claims. DeepSeek-V2 and DeepSeek-V3 make MLA part of the trained architecture, while a runtime must still preserve the latent representation through cache allocation and a compatible attention backend [25, 32, 29, 60].

The practical question is therefore not whether the latent tensor is smaller. It is whether the selected serving path retains that tensor, interprets its layout correctly, and improves the workload quantity being measured. A generic backend that materializes ordinary keys and values may surrender the storage advantage; a specialized backend can consume latent-plus-RoPE pages directly. Neither choice determines model quality from cache arithmetic alone.

Remark 4.7.1 (Limits of MLA) MLA changes the payload retained for each position. Sparse attention changes which positions enter an attention calculation. A serving claim additionally needs the workload, runtime, kernel path, dtype, batch shape, hardware, and measurement boundary.

Table 4.5 keeps the resulting decision compact. The complete definitions of MHA, MQA, GQA, MLA, and sparse selection remain in the preceding sections; this table states only what must be measured when choosing among them.

Retained local records support one bounded consequence: a runtime can preserve the latent-plus-RoPE representation, and equivalent synthetic cache-read paths can scale differently with sequence length. They do not establish physical DRAM traffic, general batching behavior, model quality, or an end-to-end speedup. In Volume II, the FlashMLA chapter owns the operator boundary, the serving chapter owns the request path, and the NVIDIA Kernels and Runtime Measurements chapter owns the RTX 5090 and H200 campaign tables and provenance.

Consider two runtimes serving the same checkpoint and requests. Runtime A allocates latent-plus-RoPE pages and gives them directly to an MLA-aware backend. Runtime B expands every appended latent row into ordinary per-head keys and values because its attention backend understands only materialized

Table 4.5.: Cache and attention choices change different resources. The required evidence prevents a shape result from becoming an unsupported serving claim.

Candidate	Resource effect	Evidence needed for a serving choice
Materialized MHA	Ordinary per-head KV state	Cache bytes, quality, latency, and selected kernel for the same workload
GQA or MQA	Fewer stored KV heads	The same workload and a declared head-sharing layout
MLA	Smaller latent-plus-positional payload	Public dimensions, retained layout, backend selection, correctness, memory, and decode timing
Sparse attention	Fewer scored positions	Selector identity, recall or quality, latency, and the task consequence of omitted positions

MHA. Both can preserve model outputs within tolerance, but only A preserves the architectural cache-byte advantage. This example is why checkpoint identity and correctness are necessary yet insufficient coordinates for a serving-memory claim.

4.8. MLA Versus Sparse Attention

MLA and sparse attention address different axes of the same decode problem. MLA changes what the model stores for each previous position. Sparse attention changes which previous positions are selected for scoring and aggregation. The distinction is essential because both ideas can appear in the same long-context system, and because a kernel name that mentions MLA can still have dense or sparse modes. FlashMLA, for example, documents dense MLA decoding kernels and sparse attention kernels. In this context, the terminology separates representation compression from position selection [60].

Definition 4.8.1 (Representation compression) *Representation compression changes the stored payload $C_{r,\ell,t}$ associated with request r , layer ℓ , and retained logical position t , reducing its size or layout while keeping the set of positions available to the attention calculation fixed.*

Definition 4.8.2 (Position sparsity) *Position sparsity changes the selected-position set $\mathcal{J}_{r,\ell,i} \subseteq \mathcal{D}_{r,i}$ that query position i scores or aggregates over, while leaving open what payload $C_{r,\ell,t}$ is stored for each selected position.*

Definitions 4.8.1 and 4.8.2 separate two independent axes in the running example. With four previous positions, dense MHA attends to positions 1, 2, 3, 4 and stores ordinary per-head keys and values for each of them. Dense MLA still attends to positions 1, 2, 3, 4, but stores (c_i, k_i^R) for each position. A sparse dense-cache system might attend only to positions 2 and 4, while storing ordinary keys and values for the positions it may select. A sparse MLA system might also attend only to positions 2 and 4, but read latent entries for those positions. These four cases differ along two independent axes.

The order of questions matters. First ask which positions the query may use; then ask what representation is stored for each usable position. Reversing those questions is the common category error: a smaller latent payload does not itself remove positions, and a sparse position set does not determine whether each retained entry is dense or latent.

Figure 4.7 puts the same four cases on those axes: cache payload changes left to right, while selected positions change top to bottom.

Table 4.6.: Dense and sparse attention axes separated from cache representation.

case	positions for x_5	state per selected position
dense MHA	$\{1, 2, 3, 4\}$	(K_i, V_i) per KV head
dense MLA	$\{1, 2, 3, 4\}$	(c_i, k_i^R)
sparse dense cache	$\{2, 4\}$	(K_i, V_i) per KV head
sparse MLA	$\{2, 4\}$	(c_i, k_i^R)

Figure 4.8.: Attention-efficiency ladder under the chapter’s toy dimensions. MHA, GQA, and MQA vary stored KV-head sharing; MLA changes the representation stored per retained position; sparse MLA additionally changes the position set. The arrows organize comparison questions rather than claiming inheritance or wall-clock speed ratios.

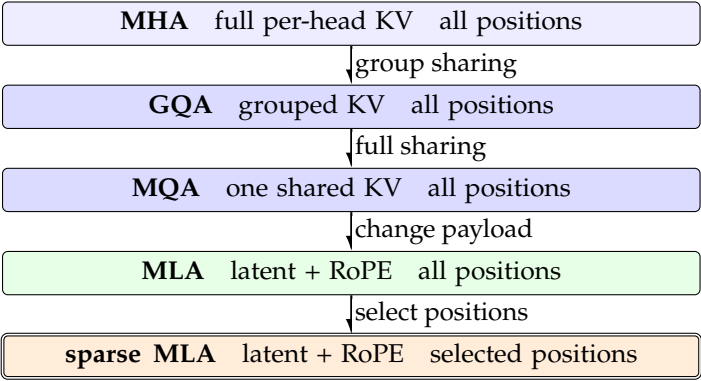


Figure 4.7.: MLA changes the payload stored for each selected position, while sparse attention changes which positions are selected for scoring and aggregation.

	full KV payload	latent payload
all positions 1, 2, 3, 4	dense MHA	dense MLA
selected subset e.g. 2, 4	sparse dense	sparse MLA

The numerical effect follows from the axes. In the toy dimensions, dense MHA reads $4 \cdot 16 = 64$ scalars. Dense MLA reads $4 \cdot 4 = 16$ scalars. Sparse dense-cache attention over two selected positions reads $2 \cdot 16 = 32$ scalars. Sparse MLA over two selected positions reads $2 \cdot 4 = 8$ scalars. The comparison is not a recommendation that sparse MLA is always best; it is a dimensioned explanation of what each axis saves under the stated assumptions.

Attention-Efficiency Ladder The chapter’s comparisons can now be read as one sequence of design questions. Can query heads share stored key/value heads? Can the stored payload itself be replaced by a latent representation? Can the main attention path read only a selected subset of positions? Figure 4.8 places those questions on one ladder. It is a conceptual comparison, not an architecture-inheritance claim or a ranking of model quality and wall-clock speed. The learned selection mechanism, its training objective, and its recall obligation belong to Chapter 11, where DeepSeek Sparse Attention instantiates the final axis.

Read the ladder by tracking two quantities. MHA, GQA, MQA, and MLA keep all four earlier positions visible while progressively sharing or changing the stored payload. MQA and MLA reach the same toy payload size by different representations: MQA stores ordinary shared KV, while MLA stores latent and

rotary state. Sparse MLA also changes the number of selected positions. The exact two-axis arithmetic appears once in the paragraph immediately above the ladder. This separation also protects the chapter's scope. The present chapter teaches the representation axis because it is needed before reading the DeepSeek-V3 architecture and the later serving chapters. The details of token-level sparse attention, sparse-index formats, and long-context retrieval belong in the later long-context and FlashMLA chapters. Here, the only necessary point is that compressed state and sparse position selection are different decisions.

4.9. Concept-to-Code Map

The mathematical objects in this chapter become inspectable through the public DeepSeek-V3 configuration and model implementation. The configuration names the MLA dimensions; the forward path shows the query split, latent-KV state, and RoPE-sensitive state. Those files establish the model object. They do not by themselves establish that a serving runtime allocates, pages, appends, and reads the same representation [29].

Concept-to-code map

The MLA projection correspondence follows <https://github.com/deepseek-ai/DeepSeek-V3/blob/main/inference/model.py>.

- **MLA architecture claim.** The chapter's MLA concept maps to the model record in the DeepSeek-V3 README.
- **MLA dimension symbols.** H_Q , r , d_{nope} , d_{rope} , and d_v map to `n_heads`, `kv_lora_rank`, `qk_nope_head_dim`, `qk_rope_head_dim`, and `v_head_dim`.
- **Projection and cache state.** The model-side projection, split, and retained objects map to `MLA.forward`, `kv_cache`, and `pe_cache`.

The implementation counterparts of Definitions 4.3.1, 4.5.2, and 4.4.1 are therefore specific: the forward path constructs non-rotary and rotary query components, stores latent and positional cache state, and can score against the absorbed representation. A model-support claim must preserve those shapes and pairings.

The physical realization has three later owners. First, the serving chapter in Volume II follows the configuration through model registration, cache allocation, append/read state, request ownership, and reference tests. Second, the FlashMLA chapter in Volume II instantiates the kernel boundary with documented metadata, cache layouts, and decode entry points [60]. Third, the systems correctness chapter in Volume II checks that physical traversal reconstructs the same logical positions and output as the model-level reference.

The boundary is deliberate: this chapter proves why latent-plus-RoPE state is the model-visible representation. The later chapters prove whether a selected runtime and kernel preserve it. Parsing the five configuration fields is necessary for MLA support, but it is not evidence of correct cache allocation, backend selection, or serving behavior.

Read the public forward path in this order: configuration fixes the head and latent dimensions; model construction allocates the matching projections; `MLA.forward` forms and splits the query, compresses the current KV state, applies the rotary

branch, and appends latent and positional cache state; the score path then consumes the paired representation and reconstructs the attention output. The named fields are therefore not an unordered checklist. Their producer-consumer order is the code-level counterpart of the chapter's shape derivation.

4.10. Summary

MLA changes the representation stored in the attention cache. MHA stores per-head key and value vectors. MQA and GQA reduce cache state by sharing key/value heads across query heads. MLA instead stores a low-rank latent KV state that supports the non-RoPE score and value-reconstruction paths, together with the RoPE-sensitive component needed for positional scoring. DeepSeek's source name NoPE identifies only those non-RoPE query/key coordinates; it does not mean that the model as a whole omits positional encoding. Table 4.3 centralizes the exact running-example arithmetic. The precise savings in a real model depend on the public dimensions, dtype, layout, and backend, but the representation distinction is mathematical and source-visible.

Runtime support has to connect configuration, projection paths, cache allocation, append/read layout, and attention kernels. The filled decode handoff adds the ownership check: logical positions, physical cache locations, append destination, representation metadata, and output row must stay aligned even when every tensor keeps a legal shape. Sparse attention is a separate axis: it chooses positions, whereas MLA changes the state stored for positions. That distinction prepares the reader for Chapter 5 in Volume I and, in Volume II, the serving-runtime chapter, the long-context chapter, and the FlashMLA chapter.

4.11. Exercises

Use Invariant 4.5.3 when comparing positional and latent-cache state.

1. CORE: With $H_Q = 4$, $d_k = d_v = 2$, and $T = 4$, compute scalar KV-cache elements read by one MHA layer during decode, then convert to bf16 bytes.
2. CORE: Repeat the cache-element calculation for GQA with $H_{KV} = 2$, where heads 1, 2 share the first KV head and heads 3, 4 share the second.
3. CORE: For an MLA cache entry (c_t, k_t^R) with $r = 3$ and $d_{\text{rope}} = 1$, compute the per-token cached scalar elements and compare the total read count at $T = 4$ with the MHA and GQA answers.
4. CORE: Define cache read volume in dtype bytes. Apply it to an MQA cache with $H_{KV} = 1$, $d_k = d_v = 2$, $T = 4$, and bf16 storage.
5. APPLIED: Explain MLA's non-RoPE and RoPE score branches. Identify their latent-projection and position-sensitive components, and relate "non-RoPE" to DeepSeek's "NoPE" without implying model-wide absence of positional encoding. Run `chapter_multi_head_latent_attention/nope_rope_score_checkpoint.py`, evaluate `split_score(0)`, and explain why its three returned components preserve the chapter's positional-scoring invariant.

6. APPLIED: Map these DeepSeek-V3 public fields to the chapter's symbols: `kv_lora_rank`, `qk_nope_head_dim`, `qk_rope_head_dim`, and `v_head_dim`. Use r , d_{nope} , d_{rope} , and d_v .
7. CORE: With a 2×3 matrix W , a two-coordinate query q , and a three-coordinate latent vector c , show how $q^\top(Wc) = (W^\top q)^\top c$. State every shape.
8. CORE: Why can parsed MLA fields still require specialized kernels? Give two reasons.
9. APPLIED: Contrast compressed full attention with uncompressed sparse attention; name one system for each.
10. RESEARCH: Record one MLA artifact's path, object, and citation.